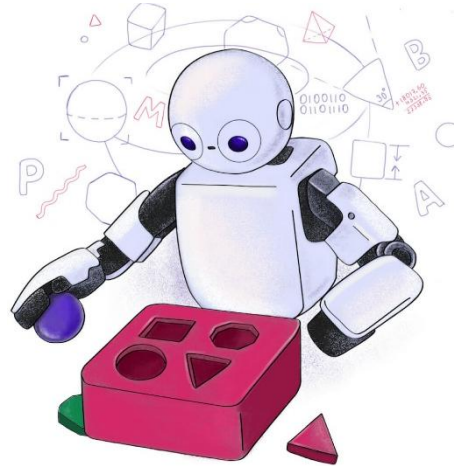
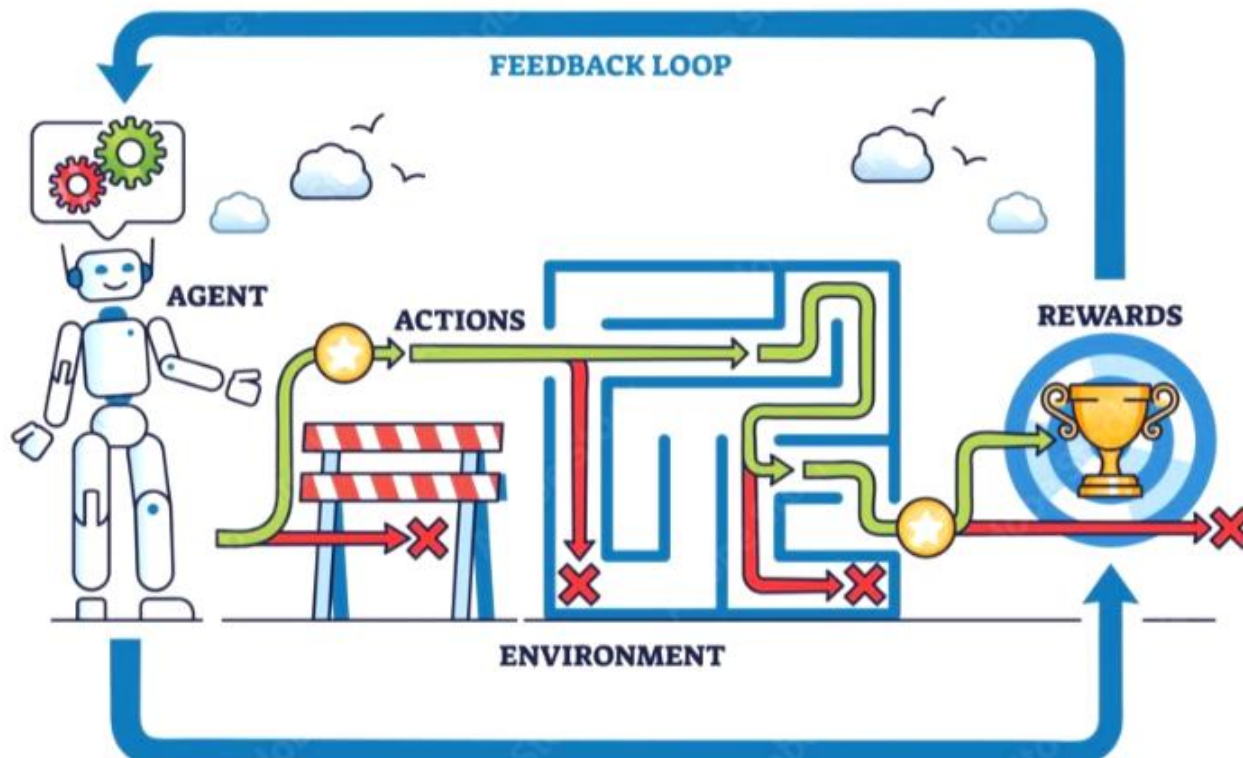


TP558 - Tópicos avançados em Machine Learning: *Q-Learning*



Contexto

- Um **agente** aprende em um **ambiente** desconhecido, sem mapa e sem respostas prontas.
- Ele age, observa **recompensas** e ajusta suas decisões por tentativa e erro.
- O objetivo é **aprender** a tomar as melhores decisões apenas interagindo com o mundo.
- No **aprendizado por reforço**, não há dados pré-rotulados: o feedback vem da **interação**.



Introdução

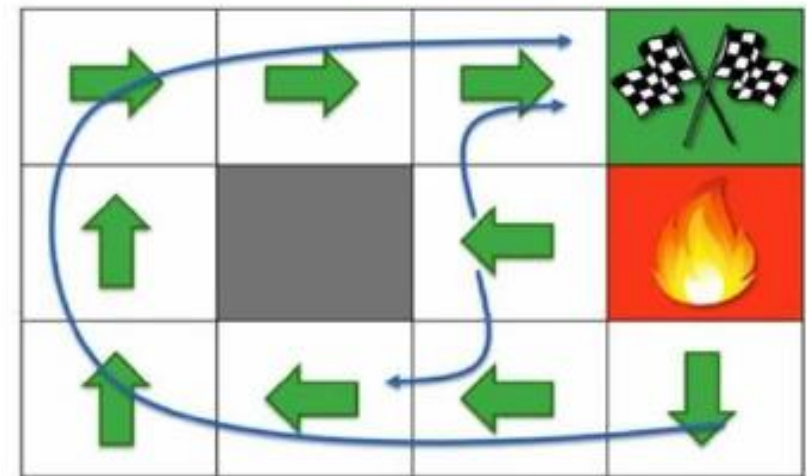
- Q-Learning é um algoritmo de aprendizado por reforço **model-free** e **off-policy**.
- O agente aprende uma **política ótima** por tentativa e erro em **ambientes** desconhecidos ou estocásticos.
- Aplicações: jogos, controle de robôs, veículos autônomos e otimização de sistemas complexos.



Introdução

- A base do método é a **Função Q**: valor esperado de cada par **estado-ação** $Q(s, a)$.
- Em sua forma clássica, esses valores são armazenados em uma **Q-table**.
- A cada interação, o agente executa uma ação, recebe recompensa e atualiza $Q(s, a)$.
- O objetivo é maximizar a **recompensa** acumulada ao longo do tempo.

V=0.71	V=0.74	V=0.86	
V=0.63		V=0.39	
V=0.55	V=0.46	V=0.36	V=0.22



FUNDAMENTOS MATEMÁTICOS

- **Processo de Decisão de Markov (MDP):** modelo para decisões sequenciais
- **Estados (S):** situação observável do agente — posição do robô, condições do jogo, etc.
- **Ações (A):** O que o agente pode fazer — mover, pegar, soltar
- **Transições de Estado:** representação numérica do movimento de um agente de um estado $s \rightarrow s'$ ao realizar a ação A —probabilidade de chegar a um novo estado

$$P_{ss'}^a = P[S_{t+1} = s' | S_t = s, A_t = a]$$

- **Recompensas (R):** A recompensa é a informação fornecida ao agente no ambiente para que ele possa aprendê-la— Feedback — 'Parabéns, atingiu o objetivo!' (+100). Bateu na parede!' (-10).

$$R_{ss'}^a = E[R_{t+1} | S_t = s, A_t = a]$$

- **Fator de Desconto (γ):** multiplicador que diz: recompensas futuras valem menos que recompensas imediatas — a depreciação tem um valor entre 0 e 1

FUNDAMENTOS MATEMÁTICOS

- **Política:** a estratégia que o agente usa para escolher ações a partir dos estados
- Mapeia estados em ações e é ajustada ao longo do tempo, à medida que ele aprende.

$$\pi(a|s) = P[A_t = a|S_t = s]$$

- π é a probabilidade de escolher a ação a no estado s , no instante t .
- **Função de valor de estado (v_π):** o critério que determina qual política é melhor: o valor esperado de recompensa futura a partir de s , sob a política π .

$$v_\pi(s) = E_\pi[R_{t+1} + \gamma v_\pi(S_{t+1})|S_t = s]$$

- $R_{(t+1)}$ recompensa recebida no instante seguinte ($t+1$), após a ação tomada em t
- **Função de valor de ação (q_π):** considera não só o estado, mas também a ação escolhida. É a **função Q**, o critério que o agente usa para selecionar a ação.

A Equação de Bellman

- Richard Bellman mostrou que o valor de um estado pode ser decomposto em dois termos: $V(s) = \text{recompensa imediata} + \text{valor descontado do próximo estado}$.

$$v_{\pi'}(s) = \sum_{a \in A} \pi(a|s) (R_{t+1} + \gamma \sum_{s' \in S} P_{ss'}^a v_{\pi}(s'))$$

- O primeiro somatório (**sobre as ações**) representa a política de probabilidade para realizar a ação. O segundo somatório (**sobre os próximos estados**) representa a matriz de probabilidade de transição de estado.
- Em vez de calcular a soma infinita de recompensas futuras, o problema é quebrado de forma **recursiva**: o valor do estado próximo nos diz quanto vale seguir em frente.

A Equação de Bellman de Otimalidade

- Troca a média sobre as ações pelo **máximo**: o agente escolhe sempre a ação que maximiza o valor esperado. É a **política ótima** que se busca dentro do problema definido pelo MDP.

$$v_*(s) = \max_a E_{\pi} [R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s]$$

- Resumo da comparação: **Bellman de expectativa** avalia uma política dada (faz a **média** sobre as ações de π); **Bellman de otimalidade** decide (usa o **máximo**). Essa troca é a ponte para o Q-Learning: o cálculo passa a ser **iterativo** e, portanto, implementável e treinável.

A regra de atualização

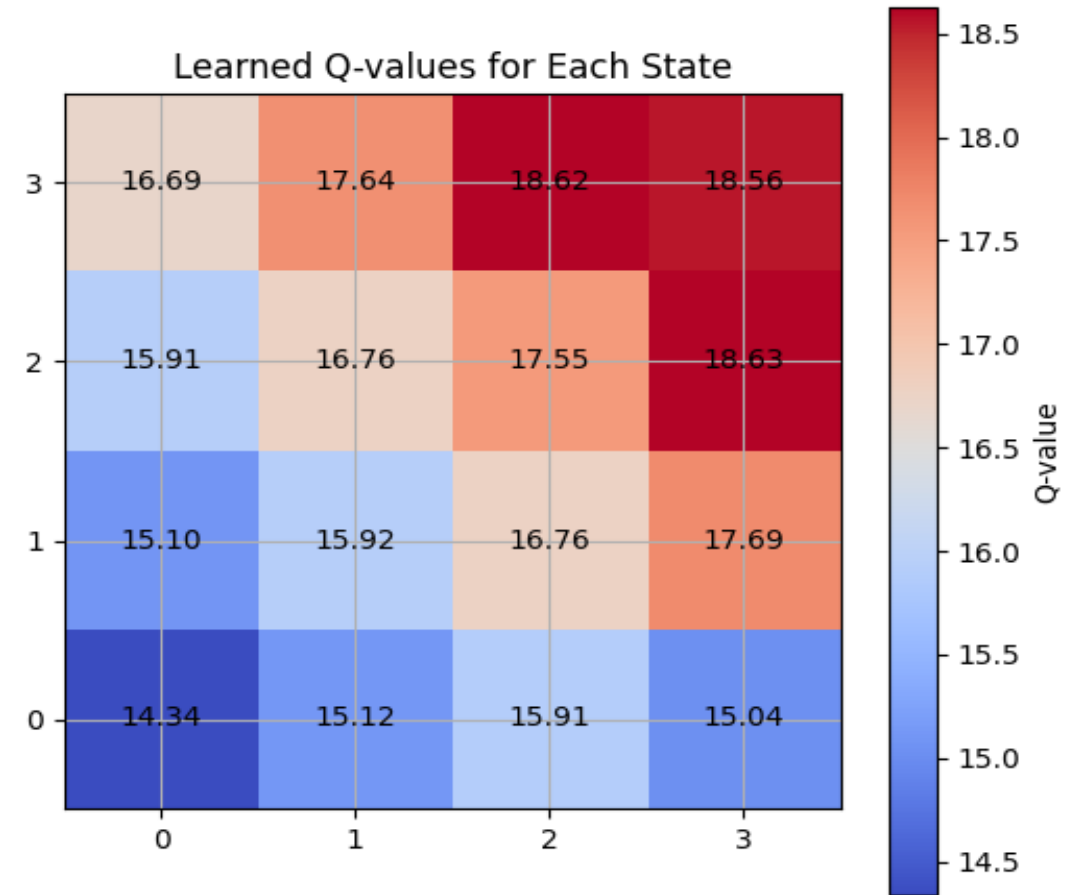
- Diferente de métodos **on-policy** como o SARSA — que atualiza usando a ação realmente escolhida pela política de comportamento — o Q-Learning usa um método **off-policy**: separa a política que executa a ação (geralmente ϵ -greedy, para explorar) da política que está sendo aprendida (a política gulosa, que maximiza Q).
- No SARSA, mesmo que a próxima ação escolhida fosse medíocre (por exploração), essa ação entrava na atualização. No Q-Learning, o alvo usa sempre o **máximo** sobre as ações do próximo estado, independentemente da ação que será de fato executada — o que evita esse viés.

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

- $Q(s, a)$ = Valor esperado de tomar ação a no estado s
- α = taxa de aprendizado — quanto do erro é incorporado em cada passo; valor entre 0 e 1 (**tipicamente 0,1**).
- $\gamma \max Q(s, a)$ — **alvo bootstrap**: a melhor estimativa disponível do valor futuro, descontada por γ .
- Erro TD (diferença temporal) — o termo entre colchetes: a distância entre o que foi **observado** e o que era **estimado**.

A Q-table

- A **Q-table** é a tabela onde o agente armazena quais ações geram as melhores recompensas em cada estado — ela representa o **entendimento do agente** sobre o ambiente, e é **atualizada continuamente** durante o aprendizado.
- As **linhas** representam os **estados**; as **colunas**, as **ações** possíveis. Cada célula corresponde ao **valor Q** de um par estado-ação. **Interpretação**: valores mais altos indicam ações melhores — as que aproximam o agente do objetivo.



Exemplo: Q-Learning

```
import numpy as np

n_estados, nacoes = 16, 4 # grid 4x4
obstaculos, meta = [5, 7, 11, 12], 15
alfa, gama, epsilon = 0.8, 0.95, 0.2

Q = np.zeros((n_estados, nacoes))

for episodio in range(2000):
    s = np.random.choice(inicios)
    for passo in range(100):
        if np.random.rand() < epsilon:
            a = np.random.randint(nacoes)
        else:
            a = np.argmax(Q[s])

        s2 = proximo_estado(s, a)
        r = recompensa(s2)

        Q[s, a] += alfa * (r + gama*np.max(Q[s2]) - Q[s, a])

        s = s2
    if terminal(s): break
```

Hiperparâmetros: taxa α , desconto γ e exploração ϵ

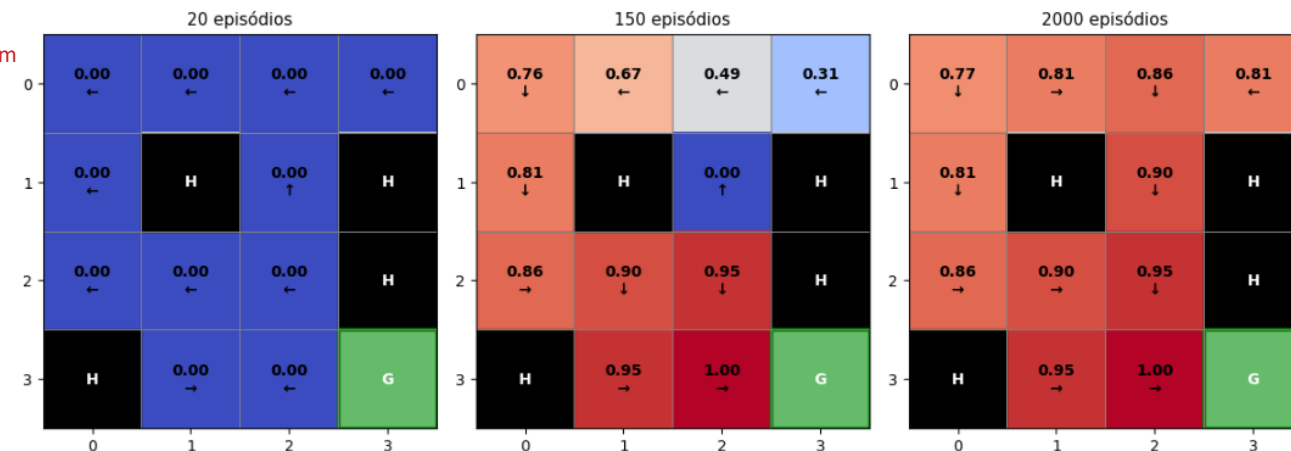
Q-table: 16 estados $\times$ 4 ações, iniciada em zero

Política ϵ -greedy: explora com prob. ϵ , senão escolhe $\arg\max Q$

Equação de Bellman: $Q \leftarrow Q + \alpha \cdot (\text{erro TD})$

Fim do episódio: caiu em H ou chegou a G

- A recompensa +1 da meta propaga-se para trás pela equação de Bellman: cada estado vale aproximadamente γ^d , sendo d o número de passos até G.
- Com poucos episódios a Q-table ainda é quase nula; com exploração suficiente a política converge e contorna todos os obstáculos.



Política aprendida em função do número de episódios de treinamento.

Vantagens e limitações

- **Simplicidade e boa performance em ambientes de agente único:** ao contrário de outros métodos, o Q-Learning é simples e apresenta excelente capacidade de aprendizado em ambientes com um único agente.
- **Limitação: atualização única por ação:** o valor é atualizado apenas uma vez por ação; em espaços grandes, muitas combinações estado-ação podem **nunca ser experimentadas**.
- **alto consumo de memória:** como a Q-table é pré-definida, exige memória de armazenamento considerável.
- **desvantagem em ambientes multiagente:** com dois ou mais agentes o espaço estado-ação cresce demais e o Q-Learning básico não realiza aprendizado eficaz.
- Por isso surgem **aproximações** e **variantes** mais avançadas.

Algoritmos para Agente Único

- **Deep Q-learning** — substitui a tabela por uma rede neural **CNN**, com experience **replay** e **target network** para estabilizar o aprendizado.
- **Hierarchical Q-learning** — divide as ações em níveis alto/baixo (**abstract actions**) para reduzir complexidade.
- **Double Q-learning** — usa duas funções Q para reduzir o **viés de superestimação**.
- Outras variantes : multistep Q-learning, Bayesian Q-learning, asynchronous stochastic Q-learning.

Algoritmos Multiagente

- **Modular Q-learning** — decompõe o problema grande em submódulos e integra decisões.
- **Ant Q-learning** — agentes cooperam compartilhando valores de recompensa.
- **Nash Q-learning** — estende $Q(s,a)$ para $Q(s,a_1...a_n)$; busca o equilíbrio de Nash entre agentes.
- **Swarm-based Q-learning** — combina Particle Swarm Optimization com Q-learning para convergir mais **rápido**

Aplicações

- **Jogos e Simulação** (AlphaGo , Atari)
- **Robótica Autônoma** (Robôs aprendem a andar, evitar obstáculos, subir escadas)
- **Energia** (Microgrids inteligentes: Equilibrar demanda com produção solar/eólica. Recarga de veículos elétricos: Aprender padrões de demanda)
- **Saúde** (Tratamento de câncer: Determinar sequência e doses de quimioterapia, Controle de diabetes: Ajustar insulina em tempo real)
- **Redes e Comunicações** (Roteamento: Decidir melhor caminho para pacotes em redes congestionadas. Alocação de recursos: Em redes 5G, Q-Learning otimiza uso de espectro)

Conclusão

- Q-Learning é um algoritmo off-policy que aprende a melhor ação para cada estado.
- Suas variantes atendem problemas de agente único e multiagente.
- Métodos avançados podem ser desnecessários em problemas simples, mas superam o Q-Learning básico em ambientes complexos.
- Com o avanço da Inteligência Artificial, o Q-Learning segue como base para sistemas inteligentes.

Perguntas?

Referências

- B. Jang, M. Kim, G. Harerimana and J. W. Kim, "Q-Learning Algorithms: A Comprehensive Classification and Applications," in IEEE Access, vol. 7, Sep, 2019, pp. 133653-133667.
- Data Science Academy. Deep Learning Book, 2025. Disponível em: <https://www.deeplearningbook.com.br/>.

Quiz

<https://forms.gle/oUpPcLaf7JKeHW917>

Obrigado!